Week 47

v1.0.1

Table of contents

1 Determination of reaction order and activation energy 2
2 The pH effect on an active site histidine residue 10
3 Qualitative description of Michaelis-Menten kinetics 13
4 Design of an enzyme kinetics experiment 14
5 Lab Exercise Analysis: Michaelis Menten 23
6 Analysis of a data set obeying Michaelis-Menten kinetics 30
7 The Michaelis Menten equation 34

```
try:
    import fysisk_biokemi
    print("Already installed")
except ImportError:
    %pip install -q "fysisk_biokemi[pyodide] @ https://au-mbg.github.io/fysisk-biokemi/
wheels/fysisk_biokemi-0.2.2-py3-none-any.whl"
```

---

1 Determination of reaction order and activation energy

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from IPython.display import display
from scipy.optimize import curve_fit
pd.set_option('display.max_rows', 6)
```

The irreversible isomerization of compound A to compound B results in a decreasing absorbance. The isomerization was followed in a time course at two different temperatures (T1 = 25 °C and T2 = 40 °C). The absorbance (ϵ = 16700 $cm^{-1}M^{-1}$) was used to calculate the concentration of compound A in a spectrophotometer with a pathlength of 1 cm. The obtained dataset is given in the file `deter-reacti-order-activ.csv`.

(a) Temperatures

What are the two temperatures in Kelvin? Set them as varilabes in the cell below.

```
## Your task: Convert the temperatures to K and assign to the variables:
T1 = 25 + 273.15
T2 = 40 + 273.15
```

(b) Load the dataset

Load the dataset with pandas. Set `data_file_path` to the path of the dataset file named above.

```
from fysisk_biokemi.datasets import load_dataset
df = load_dataset('reaction_order_activation_week48') # Load from package for the solution
so it doesn't require to interact.
display(df)
```

	t_(s)	Abs(t)_25C	Abs(t)_40C
0	0.000	0.835	0.835
1	0.002	0.801	0.806
2	0.004	0.787	0.731
...	...	...	...
23	0.046	0.327	0.207
24	0.048	0.321	0.201
25	0.050	0.306	0.185

(c) SI Units

Calculate the concentration of A at each timepoint in SI units, by adding new columns to the `DataFrame`.

```
## Your task: Assign known values ##
extinction_coeff = 16700
```

```
L = 1 # Path length

## Calculate concentrations using the variables above and the data in the dataframe ##
df['[A]_(M)_25C'] = df['Abs(t)_25C'] / (extinction_coeff * L)
df['[A]_(M)_40C'] = df['Abs(t)_40C'] / (extinction_coeff * L)
display(df)
```

	t_(s)	Abs(t)_25C	Abs(t)_40C	[A]_(M)_25C	[A]_(M)_40C
0	0.000	0.835	0.835	0.000050	0.000050
1	0.002	0.801	0.806	0.000048	0.000048
2	0.004	0.787	0.731	0.000047	0.000044
…	…	…	…	…	…
23	0.046	0.327	0.207	0.000020	0.000012
24	0.048	0.321	0.201	0.000019	0.000012
25	0.050	0.306	0.185	0.000018	0.000011

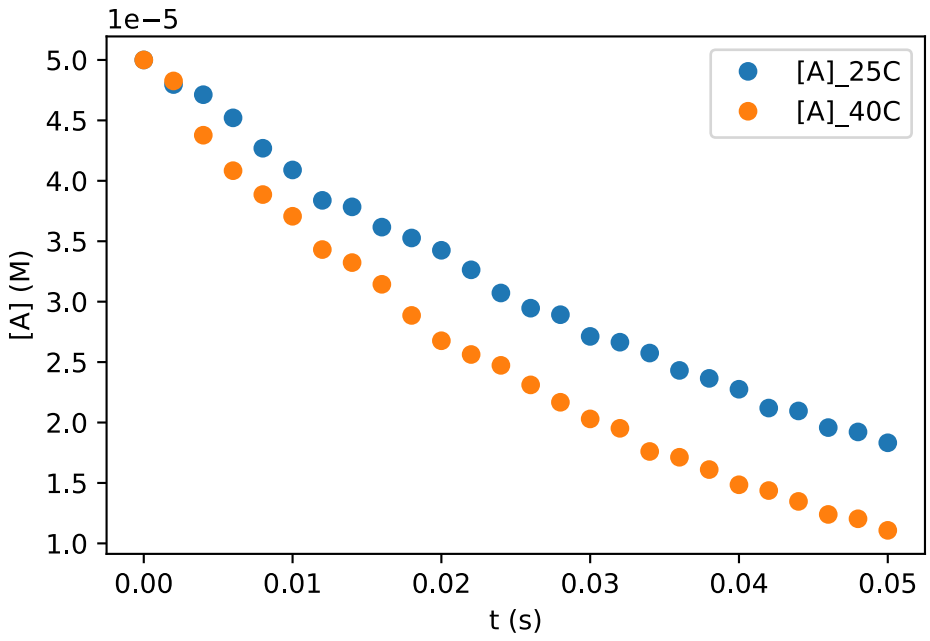
(d) Plot the data

We will be resuing the plot, so we will put the code for it in a function.

```
fig, ax = plt.subplots()

# Task: Plot the two data series in a scatter plot
# That is: t_(s) vs [A]_(M)_25C
# and: t_(s) vs [A]_(M)_40C
ax.plot(df['t_(s)'], df['[A]_(M)_25C'], 'o', label='[A]_25C')
ax.plot(df['t_(s)'], df['[A]_(M)_40C'], 'o', label='[A]_40C')

## EXTRA: This sets x and y-axis labels and shows the legends.
ax.set_xlabel('t (s)')
ax.set_ylabel('[A] (M)')
ax.legend()
```



(e) Fitting with rate equations

Now want to fit the data using integrated rate laws - in order to ultimately determine the reaction orders of the two datasets.

To do so we need functions for zeroth, first and second order integrated rate laws.

```
def zeroth_order(t, k, A0):
    ## Your task: Implement the zeroth order rate equation ##
    result = A0 - k*t
    return result

def first_order(t, k, A0):
    ## Your task: Implement the first order rate equation ##
    result = A0 * np.exp(-k*t)
    return result

def second_order(t, k, A0):
    ## Your task: Implement the second order rate equation ##
    result = A0 / (1 + 2*k*t*A0)
    return result

## Use these to check that your functions are correct!
print('zeroth_order', zeroth_order(1, 1, 2)) # Should give 1
print('first_order', first_order(1, 1, 2)) # Should give 0.7357...
print('second_order', second_order(1, 1, 2)) # Should give 0.4
```

```
zeroth_order 1
first_order 0.7357588823428847
second_order 0.4
```

Having defined these functions we can use them for fitting

```
## Task: Choose one of the rate equations and assign to the variable
rate_equation = first_order

## Task: Fit to the data at 25 C
fitted_parameters_25C, trash = curve_fit(rate_equation, df['t_(s)'], df['[A]_(M)_25C'])

## Task: Fit to the data at 40 C
fitted_parameters_40C, trash = curve_fit(rate_equation, df['t_(s)'], df['[A]_(M)_40C'])

## Extracts & prints the parameters
k_25C, A0_25C = fitted_parameters_25C
k_40C, A0_40C = fitted_parameters_40C

print(k_25C, A0_25C)
print(k_40C, A0_40C)
```

```
20.081750693534495 5.024371022380773e-05
29.96091632105259 4.990085394734088e-05
```

Having made the fits we plot it together with the two data series.

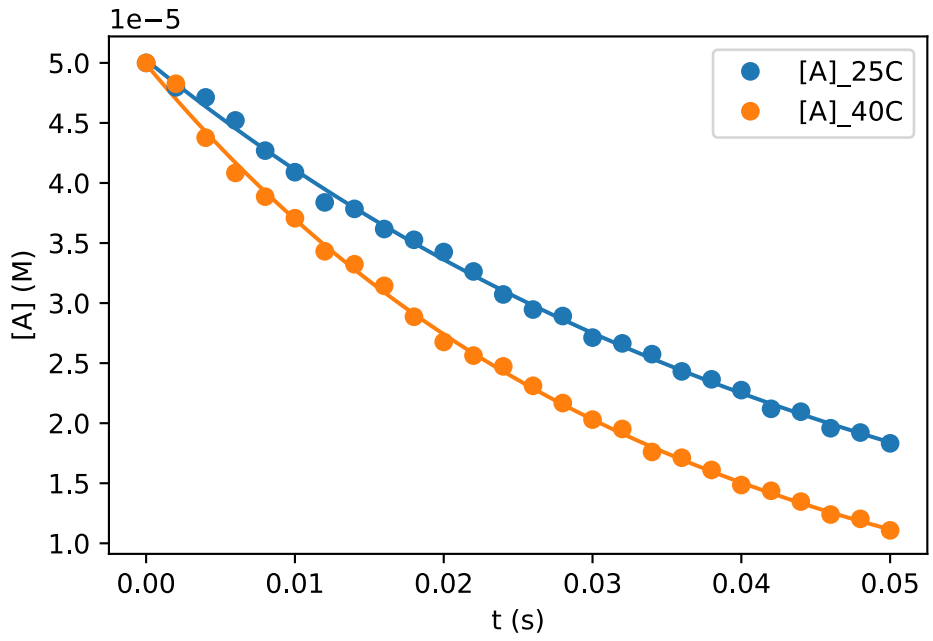
```
## Your task: Calculate the fits using the chosen rate equation &
## the found parameters found during fitting.
t_smooth = np.linspace(0, 0.05, 100)
A_fit_25C = rate_equation(t_smooth, k_25C, A0_25C)
A_fit_40C = rate_equation(t_smooth, k_40C, A0_40C)

fig, ax = plt.subplots()

## Your task: Plot the two fits
ax.plot(t_smooth, A_fit_25C)
ax.plot(t_smooth, A_fit_40C)

## This plots the data sets
ax.plot(df['t_(s)'], df['[A]_(M)_25C'], 'o', label='[A]_25C', color='C0')
ax.plot(df['t_(s)'], df['[A]_(M)_40C'], 'o', label='[A]_40C', color='C1')

## EXTRA: Sets the axis labels and legend.
ax.set_xlabel('t (s)')
ax.set_ylabel('[A] (M)')
ax.legend()
```



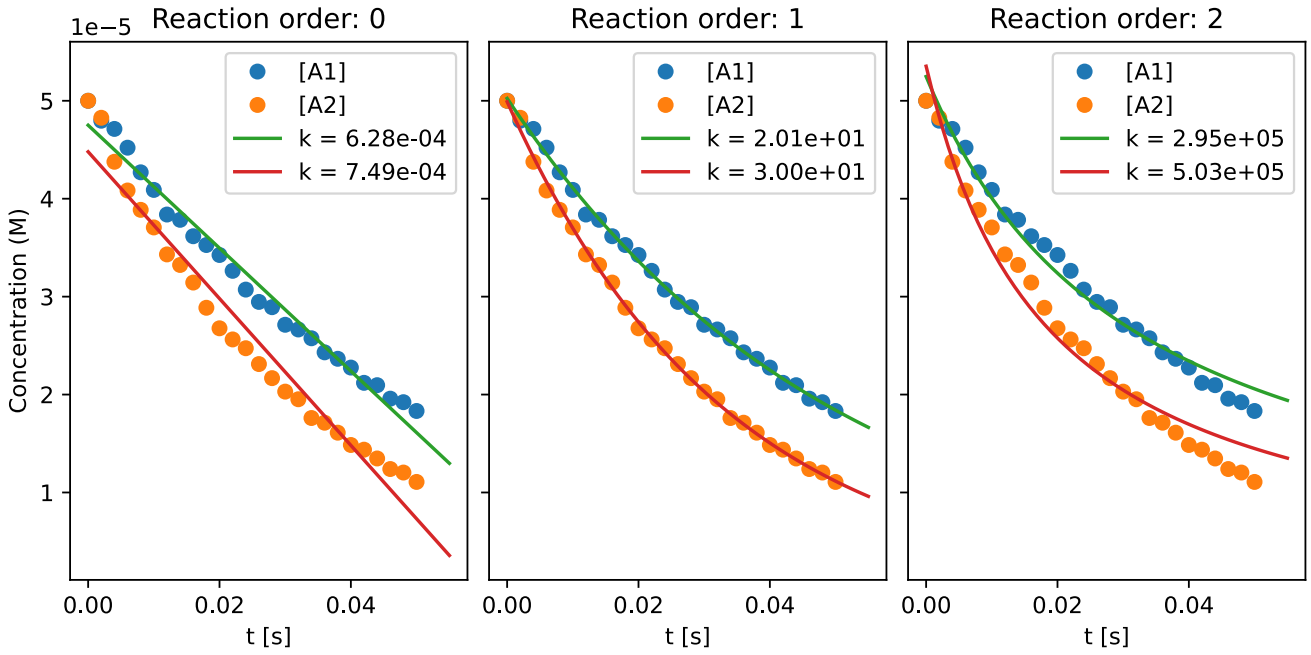
You can go back to the cell where you set `rate_equation` and use another of the three: `zeroth_order`, `first_order`, `second_order`.

(f) Determine reaction order

Based on your analysis above or by using the plot the created by the next cell determine the reaction order and the units of the rate constant for each of the two reactions.

The function makes the same fits that you made, but makes them all at once and plots the results side by side.

```
from fysisk_biokemi.utils.deter_reacti_order_activ import make_plot
rate_laws = {0: zeroth_order, 1: first_order, 2: second_order}
make_plot(df, rate_laws)
```



i Information on 'make_plot'

The function 'make_plot' is beyond what is expected for this course, however if you are interested you can see the source code for it on github.

! Answer

Both are first order reactions for which the units of the rate constant is $\frac{1}{\mathrm{s}}$.

(g) Activation energy

With the assumption that the Arrhenius constant A and the activation energy are temperature independent in the interval measured, use the Arrhenius equation to calculate the activation energy of the isomerization of the compound A.

You can use $R = 8.314 \times 10^{-3}\ \frac{\mathrm{kJ}}{\mathrm{mol} \cdot \mathrm{K}}$

Caution

You need to derive the correct equation **before** using Python to calculate the result.

Perform the calculation in the cell below.

```
## Your task: Define known & fitted values
k1 = 20.08175
k2 = 29.96091
R = 8.318 * 10**(-3) # kJ / (mol K)
```

```
## Your task: Calculate the activation energy
## Remember: Derive the correct formula first using pen and paper!
activation_energy = R * np.log(k1/k2)/(1/T2 - 1/T1) # kJ/mol
print(activation_energy)
```

```
20.71400117402973
```

! Answer

The Arrhenius equation is

$$k = A \cdot \exp\left(\frac{-E_A}{RT}\right)$$

We have this for the two situations

$$k_1 = A \cdot \exp\left(\frac{-E_A}{RT_1}\right)$$
$$k_2 = A \cdot \exp\left(\frac{-E_A}{RT_2}\right)$$

Giving us two equations with two unknowns A and E_A where E_A is the activation energy that we are interested in.

Taking the log on both sides we get

$$\ln(k) = \ln(A) - \frac{E_A}{RT}$$

Where we can isolate for $\ln(A)$

$$\ln(A) = \ln(k) + \frac{E_A}{RT}$$

We can equate this for both our equations

$$\ln(k_1) + \frac{E_A}{RT_1} = \ln(k_2) + \frac{E_A}{RT_2}$$

Rearranging a bit we get

$$\ln(k_1) - \ln(k_2) = \frac{E_A}{R}\left(\frac{1}{T_2} - \frac{1}{T_1}\right)$$

Isolating for E_A

$$E_A = R\frac{\ln(k_1) - \ln(k_2)}{\left(\frac{1}{T_2} - \frac{1}{T_1}\right)}$$
$$= \frac{R\ln\left(\frac{k_1}{k_2}\right)}{\left(\frac{1}{T_2} - \frac{1}{T_1}\right)}$$

---

2 The pH effect on an active site histidine residue

Assume that a compound R can react with the unprotonated form of Histidine, His, to form a covalent reaction product, P:

$$\mathrm{His} + \mathrm{R} \rightarrow \mathrm{P}$$

The protonated form of Histidine, HisH^+, is in equilibrium with His:

$$\mathrm{HisH}^+ \rightleftharpoons \mathrm{His} + \mathrm{H}^+$$

The pKa value for this acid-base equilibrium is 6.0. Further assume that the total concentration of Histidine, is

$$[\mathrm{HisH}^+] + [\mathrm{His}] = 10^{-3}\ \mathrm{M}$$

(a) Percentage unprotonated

What percentage of Histidine is unprotonated at pH 6.0. What do you notice about the relationship between pH and pKa in this case? (You should not need to do any arithemtic or algebra to answer this exercise.)

```
## Your task: What percentage is unprotonated?
percentage_unprotonated = 50
```

! Answer

Because pH = pKa the percentage of unprotonated is 50%, so there is no need to do a direct calculation in this case.

(b) Concentration

Determine [His] at pH 6

```
## Your task: Assign known value and calculate [His]
total_conc = 10**(-3)
his_conc = total_conc * percentage_unprotonated / 100 # Unit: M
print(his_conc)

# Or directly
his_conc = 0.5 * 10**(-3) # Unit: M
print(his_conc)
```

```
0.0005
0.0005
```

(c) Reaction order

The reaction equation for the reaction between His and R is

$$v = -\frac{d[\text{His}]}{dt} = k \cdot [\text{His}] \cdot [\text{R}]$$

What's the reaction order?

! Answer

Bimolecular so we expect second order reaction.

(d) High concentrations

If [R] is much higher than [His], what can then be concluded regarding the order of the reaction?

! Answer

$[R]$ will remain roughly constant - we are thus in the regime of a pseudo first order reaction

(e) New rate constant

Show how a new rate constant, k', can be defined in these conditions. How does k' depend on R?

! Answer

$v = k \cdot [\text{His}] \cdot [\text{R}]$

$k' = k[\text{R}]$

$v = k'[\text{His}]$

Then k' is linearly proportional with [R].

(f) SI Units

At pH 6.0 the reaction rate $v = 1\ \text{mM} \cdot \text{s}^{-1}$.

Convert the reaction rate to SI-units given in $\text{M} \cdot \text{s}^{-1}$.

```
# Your task: Calculate v in SI units M / s
reaction_rate = 1 * 10**(-3) # Units: M / s
print(reaction_rate)
```

```
0.001
```

(g) Calculate the rate constant

Use the concentration of [His] at pH 6 calculated in question (b), the reaction rate from (f) and constant $[\text{R}] = 0.2\ \text{M}$ to calculate the rate constant k.

```
## Your task: Set known value and calculate k ##
## Hint: You can use the his_conc variable ##
R_conc = 0.2
k = reaction_rate / (R_conc * his_conc)
print(k)
```

```
10.0
```

---

3 Qualitative description of Michaelis-Menten kinetics

```
import plotly
import fysisk_biokemi.widgets as fbw
from fysisk_biokemi.widgets.utils import enable_custom_widget_colab,
disable_custom_widget_colab
from IPython.display import display, HTML
enable_custom_widget_colab()
```

```
fbw.michaelis_menten_demo()
```

Use the widget created by the cell above to answer the following the questions.

(a) Change K_M

Setting $V_{\max} = 400$ and $[E]_{\text{tot}} = 0.0005$, change K_M from 0.5 to 5 to 10 to 50. Describe the change in curve appearance and explain it using the Michaelis-Menten (MM) equation.

(b) Meaning of K_M

What is the biochemical meaning of K_M?

(c) Change $V_{\max}$ and $[E]_{\text{tot}}$

Now set $[E]_{\text{tot}} = 0.0005$, and $K_M = 5$. Change $V_{\max}$ step-wise from 40 to 900. Describe the changes you observe and how they relate to the MM equation.

After this, repeat the procedure for $[E]_{\text{tot}}$ increasing it 10-fold.

(d) Substrate concentrations way below $K_M

Next, we will explore the regime where $[S] \ll K_M$. Initially, set $K_M = 10$ and zoom in on the regime from 0 to 1 on the X-axis. What does the MM equation look like in this regime? Use the MM equation and a simplifying assumption to show how this occurs.

Next, increase K_M five-fold and try to change $V_{\max}$ som the curve appearance is unchanged (beware of the autoscaling Y-axis).

Imagine an experiment conducted using only $[S]$ values from this range. How would this affect how well you could determine the MM parameters?

(f) Training estimation skills.

Train your estimation skills using the widget below.

```
fbw.michealis_menten_guess()
```

---

4 Design of an enzyme kinetics experiment

```
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
import numpy as np
import pandas as pd
from IPython.display import display
pd.set_option('display.max_rows', 6)
```

We have finally arrived at the part of the course where we are ready to analyse the data from your lab exercise in the first quarter. First, however we will analyse a similar data set where we are sure about the data quality (😉) to illustrate the process.

In the Excel document `design-enzyme-kineti-exper.xlsx` you will find a data set in which an enzyme catalyzed formation of product P, with varying start concentration of substrates, [S], was followed over time. The product absorbs light at a specific wavelength with an extinction coefficient of 0.068 $\mu\mathrm{M}^{-1} \cdot \mathrm{cm}^{-1}$, and the absorbance was measured in a light path of 1 cm throughout the time course.

Load the dataset with pandas. Set `data_file_path` to the path of the dataset file named above.

```
from IPython.display import display
from fysisk_biokemi.datasets import load_dataset
df = load_dataset('design-enzyme-kineti-exper.xlsx') # Load from package for the solution
so it doesn't require to interact.
display(df)
```

	time_(s)	Abs_S1	Abs_S2	Abs_S4	Abs_S8	Abs_S16	Abs_S32	Abs_S64	Abs_S128	Abs_S256
0	0.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
1	1.0	0.002	0.004	0.008	0.015	0.025	0.038	0.051	0.063	0.068
2	2.0	0.004	0.008	0.015	0.028	0.048	0.073	0.099	0.121	0.131
…	…	…	…	…	…	…	…	…	…	…
18	18.0	0.022	0.042	0.080	0.145	0.246	0.376	0.511	0.623	0.677
19	19.0	0.022	0.043	0.082	0.149	0.252	0.385	0.524	0.639	0.693
20	20.0	0.023	0.044	0.084	0.152	0.257	0.394	0.535	0.653	0.709

The headers, like `Abs_S1` refer to the substrate concentration so `S1` means a substrate concentration of 1 μM.

(a) Connvert extinction coefficient to SI.

Convert the extinction coefficient to units given in $\mathrm{M}^{-1} \cdot \mathrm{cm}^{-1}$ and assign it to a variable. Also assign the light path length to a variable.

```
## Your task: Assign known values in SI units.
ext_coeff = 0.068 / 10**(-6) # 1/(M cm)
l = 1 # cm
```

(b) Calculate concentrations

Using Lambert-Beers law, calculate the concentration of product, $[P]$, in M for each time series.

The cell below setups a loop calculate the concentrations for each of these current columns in the dataframe.

```
## Your task calculate the concentration for each 'Abs_SX' column and add it ##
## as a new column.
df['C_S1'] = df['Abs_S1'] / (ext_coeff * l)
df['C_S2'] = df['Abs_S2'] / (ext_coeff * l)
df['C_S4'] = df['Abs_S4'] / (ext_coeff * l)
df['C_S8'] = df['Abs_S8'] / (ext_coeff * l)
df['C_S16'] = df['Abs_S16'] / (ext_coeff * l)
df['C_S32'] = df['Abs_S32'] / (ext_coeff * l)
df['C_S64'] = df['Abs_S64'] / (ext_coeff * l)
df['C_S128'] = df['Abs_S128'] / (ext_coeff * l)
df['C_S256'] = df['Abs_S256'] / (ext_coeff * l)
```

! Alternative solution

A more advanced alternative solution is to use a for-loop like done below, however this is outside the scope of what you're going to learn in this course.

```
for s in [1, 2, 4, 8, 16, 32, 64, 128, 256]:
    df[f'C_{s}'] = df[f'Abs_S{s}'] / (ext_coeff * l)
```

And we can check that the columns we expect have been added to the `DataFrame`.

```
display(df)
```

time_(s)	Abs_S1	Abs_S2	Abs_S4	Abs_S8	Abs_S16	Abs_S32	Abs_S64	Abs_S128	Abs_S256	C_S1	C_S2	C_S4	C_S8	C_S16	C_S32	C_S64	C_S128	C_S256
0	0.0	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
1	1.0	0.002	0.004	0.008	0.015	0.025	0.038	0.051	0.063	0.068	2.941	5.882	11.765	23.529	44.118	73.529	107.353	133.235
2	2.0	0.004	0.008	0.015	0.028	0.048	0.073	0.099	0.121	0.131	5.882	11.765	22.059	41.176	70.588	113.971	152.941	189.706
…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…	…
18	18.0	0.022	0.042	0.080	0.145	0.246	0.376	0.511	0.623	0.677	3.235	6.471	12.059	22.353	37.941	57.941	78.824	96.029
19	19.0	0.022	0.043	0.082	0.149	0.252	0.385	0.524	0.639	0.693	3.235	6.324	12.059	22.206	38.529	58.676	79.559	96.912
20	20.0	0.023	0.044	0.084	0.152	0.257	0.394	0.535	0.653	0.709	3.382	6.471	12.353	22.500	38.676	59.118	80.147	97.647

(c) Plot

We focus now on the `C_S32`-column and plot it versus the time measurements.

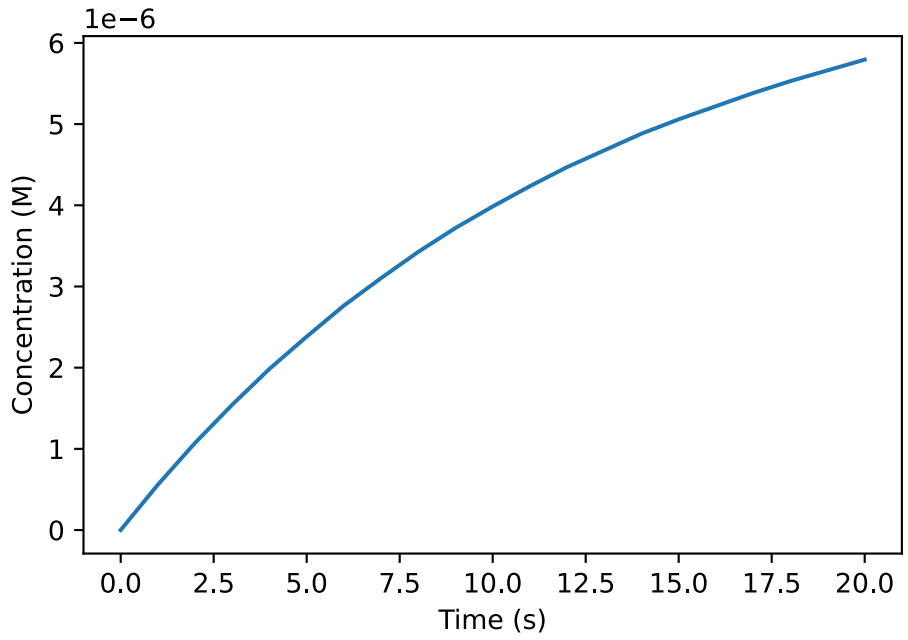
```
fig, ax = plt.subplots()

## Your task: Plot the C_S32 column
```

```
ax.plot(df['time_(s)'], df['C_S32'])

## EXTRA:
ax.set_xlabel('Time (s)')
ax.set_ylabel('Concentration (M)')
```

```
Text(0, 0.5, 'Concentration (M)')
```



(d) Extracting V_0.

In enzyme kinetics, we usually focus on the initial time points, i.e. before the reaction has progressed far enough to allow products to be accumulated and substrate to be depleted. In this early stage of the reaction, we know that the concentration has not changed significantly yet and thus the [S] = S0. Therefore, the reaction time course tends to be linear in this range.

In the following, we want to just fit the initial part of the curve. We do this by defining a new data array containing only the first values.

We want to use Python to determine V_0 for each concentration of S, in order to create a table of V_0 vs $[S]$. To do so we will fit linear functions to the initial parts of the curves, as the slope of these is then V_0.

Start by defining a linear function;

```
def linear_func(x, a, b):
    return a * x + b
```

As mentioned we want to to get the slope in the initial part of the reaction, where the curves are approximately linear. To do so we need to fit not on all the data-points but only some of them.

We will first do this for the `C_S32`-column.

```
## Determine the x and y data
x_data_full = df['time_(s)']
y_data_full = df['C_S32']

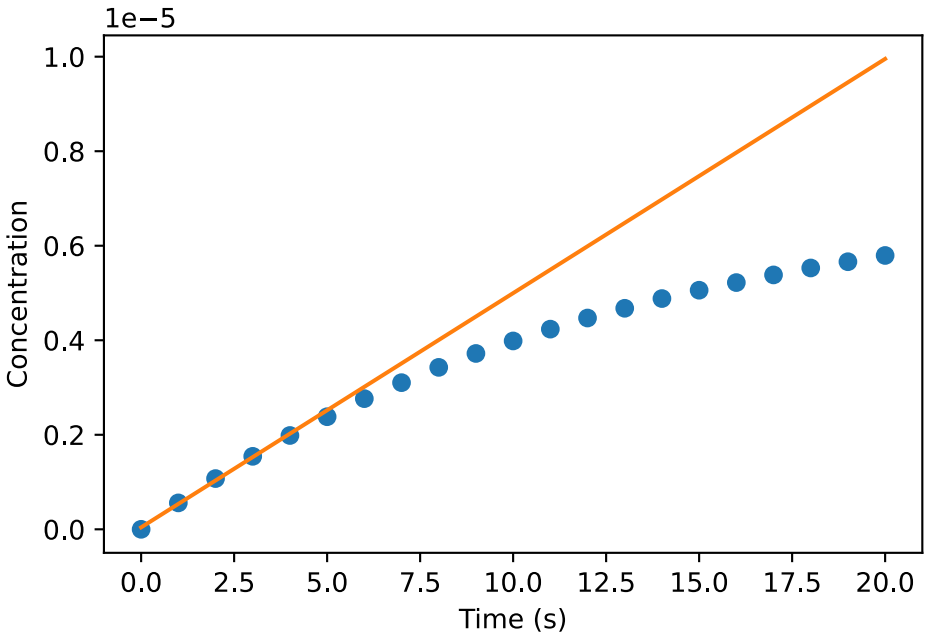
## Your task: Set n_points to choose the number of points to fit with
n_points = 5
x_data = x_data_full[0:n_points] # This picks the first n_points points.
y_data = y_data_full[0:n_points] # This picks the first n_points points.

## Your task: Make the fit.
fitted_parameters, trash = curve_fit(linear_func, x_data, y_data)
a, b = fitted_parameters
```

The cell below plots your fit alongside the data

```
fig, ax = plt.subplots()
t_smooth = np.linspace(0, 20)
y_fit = linear_func(t_smooth, a, b)
ax.plot(x_data_full, y_data_full, 'o')
ax.plot(t_smooth, y_fit)
ax.set_xlabel('Time (s)')
ax.set_ylabel('Concentration')
```

```
Text(0, 0.5, 'Concentration')
```



You want the line to fit well with the initial approximately linear part of the data, adjust `n_points` such that this is the case.

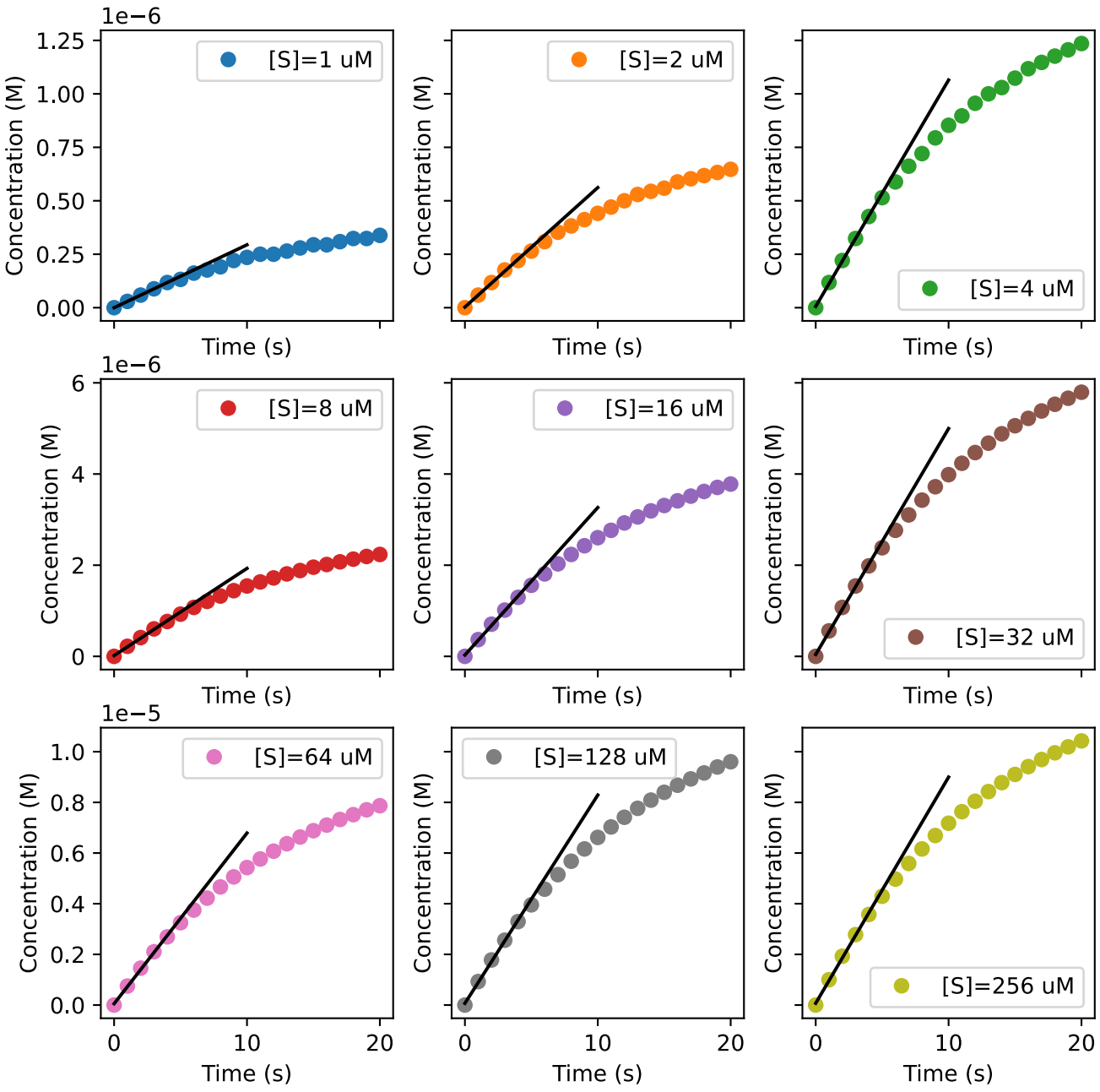
(e) Using all the datasets

In the cell below `n_points` again determines the number of data-points used for the linear fit, and makes the fits and plots for every dataset.

```
from fysisk_biokemi.utils.design_enzyme_kineti_exper import make_fits_and_plots

## Your task: Select a number of points to fit with that is appropriate in
## all cases
n_points = 5

## This function does what you did in the previous exercise but for all the datasets. ##
slopes, concentrations = make_fits_and_plots(df, n_points)
```



i Information on 'make_fits_and_plot'

The function 'make_fits_and_plots' is beyond what is expected for this course, however if you are interested you can see the source code for it on github.

Once you're satisfied with the fit you can display the collected slopes with the cell below

```
V0_df = pd.DataFrame({'[S]_(M)': concentrations, 'V0_(M/s)': slopes})
display(V0_df)
```

	[S]_(M)	V0_(M/s)
0	0.000001	2.941176e-08
1	0.000002	5.588235e-08
2	0.000004	1.058824e-07
...	...	...
6	0.000064	6.735294e-07
7	0.000128	8.220588e-07
8	0.000256	8.926471e-07

(f) Understanding of V_0

Why is it important to use V_0 rather than V at a later time point when creating the Michaelis-Menten plot?

! Answer

The initial velocities, V_0, represent velocities in which the consumption/formation is only dependent on substrate concentration as no product is yet formed.

(g) Plotting again.

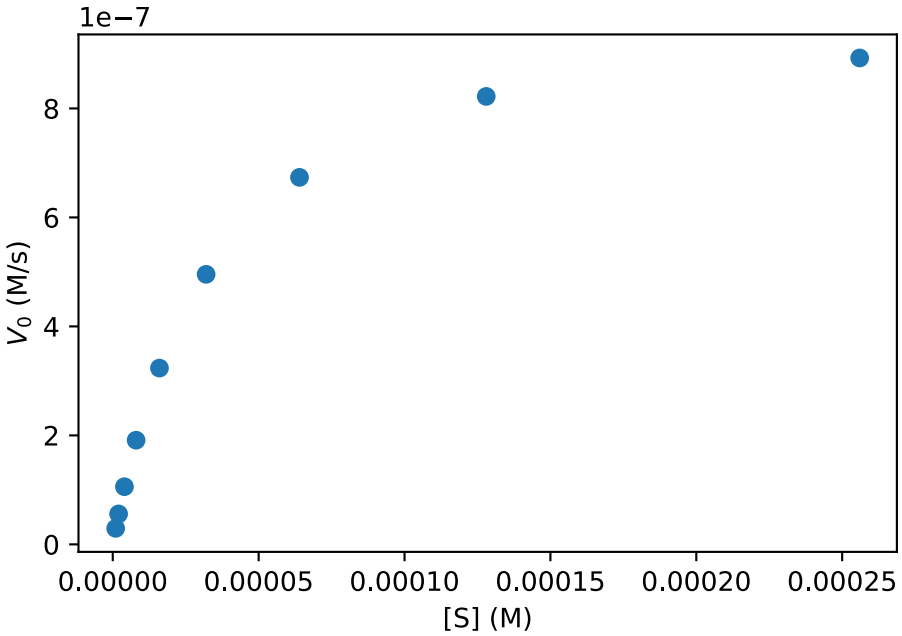
Plot V_0 against substrate concentration and estimate k_{cat} and K_M visually (remember units)

```
fig, ax = plt.subplots()

# Your task: Plot the V0_(M/s) versus [S_(M)]
ax.plot(V0_df['[S]_(M)'], V0_df['V0_(M/s)'], 'o')

## EXTRA: Axis labels.
ax.set_xlabel('[S] (M)')
ax.set_ylabel('$V_0$ (M/s)')
```

```
Text(0, 0.5, '$V_0$ (M/s)')
```



Use the figure to estimate the value of K_M and $V_{\max}$ and put those in the cell below.

```
# Your task: Put your estimated values here:
K_M_estimate = 30 * 10**(-6)
V_max_estimate = 10 * 10**(-7)
```

(h) Fitting with Michealis-Menten

Now we can fit with the Michealis-Menten equation, as always in a fitting task we need to implement the model.

```
def michaelis_menten(S, K_M, V_max):
    ## Your task: Implement the Michaelis-Menten model.
    result = (V_max * S) / (K_M + S)
    return result
```

Then use the next two cells to make the fit and plot it.

```
## Uses your estimate as initial guess
initial_guess = [K_M_estimate, V_max_estimate]
fitted_parameters, trash = curve_fit(michaelis_menten, V0_df['[S]_(M)'], V0_df['V0_(M/s)'],
initial_guess)

## Extract and prints the parameters
K_M_fit, V_max_fit = fitted_parameters
print(K_M_fit)
print(V_max_fit)
```

```
3.402856184164823e-05
1.0245295933534427e-06
```

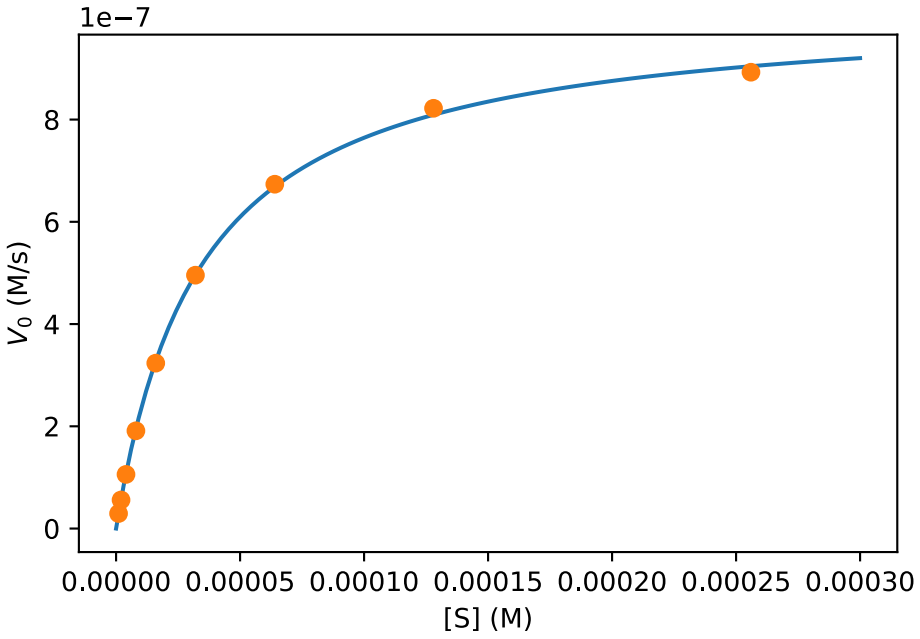
How do these compare to your estimates of the two parameters?

```
fig, ax = plt.subplots()

## Evaluates the fit
S_smooth = np.linspace(0, 0.0003, 100)
V0_fit = michaelis_menten(S_smooth, K_M_fit, V_max_fit)

## Plots the fit and the data.
ax.plot(S_smooth, V0_fit)
ax.plot(V0_df['[S]_(M)'], V0_df['V0_(M/s)'], 'o')

## Axis labels.
ax.set_xlabel('[S] (M)')
ax.set_ylabel('$V_0$ (M/s)');
```



---

5 Lab Exercise Analysis: Michaelis Menten

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
from IPython.display import display
```

In the "Introduction to the Molecules of Life" course, you performed an experiment where you measured absorption for different substrate concentrations as a function of time. The experiment was done with p-nitrophenylphosphate as the substrate, which turns into p-nitrophenol when the enzyme (alkaline phosphatase) acts on it.

If you do not have data, or you find that your data set is not of sufficient quality, you can use the following `kinetics_data.xlsx`.

Load your dataset with pandas. Set `data_file_path` to the path of your dataset file.

```
from fysisk_biokemi.datasets import load_dataset
df = load_dataset('kinetics_LØ') # Load from package for the solution so it doesn't require
to interact.
display(df)
```

	time	0.8	0.4	0.2	0.12	0.05	0.02	0.01	0.003
0	0	0.032	0.101	0.018	0.032	0.015	0.013	0.008	−0.001
1	10	0.049	0.115	0.033	0.047	0.027	0.023	0.016	0.003
2	20	0.065	0.130	0.047	0.060	0.039	0.033	0.023	0.007
…	…	…	…	…	…	…	…	…	…
16	160	0.269	NaN	NaN	NaN	NaN	NaN	NaN	NaN
17	170	0.283	NaN	NaN	NaN	NaN	NaN	NaN	NaN
18	180	0.298	NaN	NaN	NaN	NaN	NaN	NaN	NaN

(a) Absorbance to concentration

The measured absorbances are unitless, they are the log of the ratio between the incoming light intensity and the transmitted light intensity. We would like to work with concentrations instead, so we need to convert the absorbance to concentration. The conversion is done using Lambert-Beer's law.

The extinction coefficient of the product p-nitrophenol is $\varepsilon = 18000\,\mathrm{M^{-1}cm^{-1}}$ at 405 nm and the path length of the cuvette is $l = 1\,\mathrm{cm}$. We will construct a new `DataFrame` with concentrations

We will start by defining a function `lambert_beers` to calculate the concentrations

```
def lambert_beers(absorbance):
    ## Your task: Finish the implementation of LBs-law to calculate concentrations.
    L = 1 # cm
    ext_coeff = 18000 # 1 / (M cm)
```

```
    conc = absorbance / (L * ext_coeff) # M
    return conc
```

The next cell makes a new `DataFrame` by applying the lambert_beers function to every column in the original

```
data = {
    'time': df['time'],
    'S_0.8_mM': lambert_beers(df['0.8']),
    'S_0.4_mM': lambert_beers(df['0.4']),
    'S_0.2_mM': lambert_beers(df['0.2']),
    'S_0.12_mM': lambert_beers(df['0.12']),
    'S_0.05_mM': lambert_beers(df['0.05']),
    'S_0.02_mM': lambert_beers(df['0.02']),
    'S_0.01_mM': lambert_beers(df['0.01']),
    'S_0.003_mM': lambert_beers(df['0.003']),
}

df = pd.DataFrame(data)
display(df)
```

	time	S_0.8_mM	S_0.4_mM	S_0.2_mM	S_0.12_mM	S_0.05_mM	S_0.02_mM	S_0.01_mM	S_0.003_mM
0	0	0.000002	0.000006	0.000001	0.000002	8.333333e-07	7.222222e-07	4.444444e-07	–5.555556e-08
1	10	0.000003	0.000006	0.000002	0.000003	1.500000e-06	1.277778e-06	8.888889e-07	1.666667e-07
2	20	0.000004	0.000007	0.000003	0.000003	2.166667e-06	1.833333e-06	1.277778e-06	3.888889e-07
…	…	…	…	…	…	…	…	…	…
16	160	0.000015	NaN	NaN	NaN	NaN	NaN	NaN	NaN
17	170	0.000016	NaN	NaN	NaN	NaN	NaN	NaN	NaN
18	180	0.000017	NaN	NaN	NaN	NaN	NaN	NaN	NaN

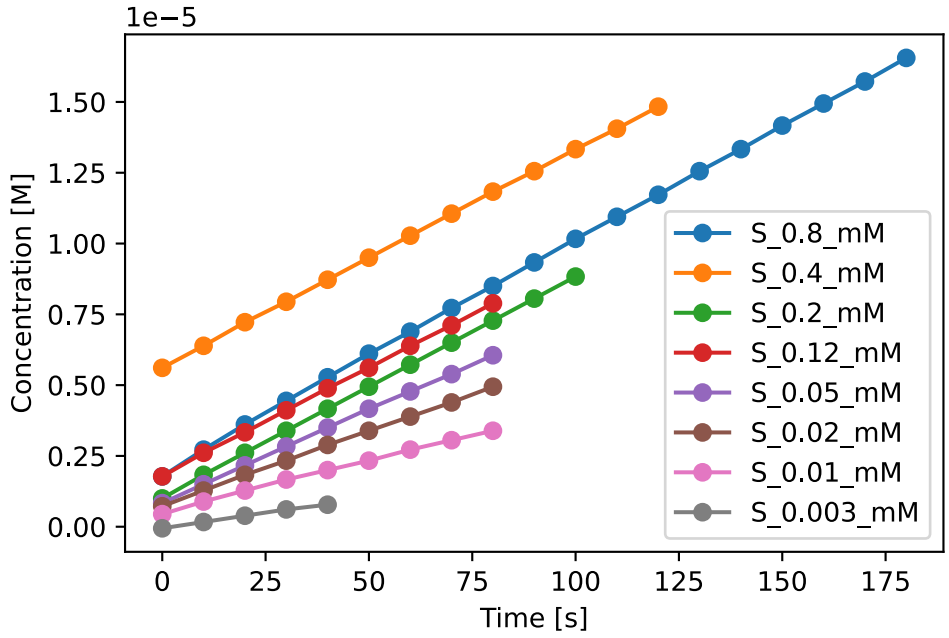
(b) Plot

Now let's plot the data. Plot each column separately as a function of time.

```
fig, ax = plt.subplots()

# Your task: Plot concentration vs. time for each column of the dataset,
# So 8 lines of code total.
ax.plot(df['time'], df['S_0.8_mM'], '-o', label='S_0.8_mM')
ax.plot(df['time'], df['S_0.4_mM'], '-o', label='S_0.4_mM')
ax.plot(df['time'], df['S_0.2_mM'], '-o', label='S_0.2_mM')
ax.plot(df['time'], df['S_0.12_mM'], '-o', label='S_0.12_mM')
ax.plot(df['time'], df['S_0.05_mM'], '-o', label='S_0.05_mM')
ax.plot(df['time'], df['S_0.02_mM'], '-o', label='S_0.02_mM')
ax.plot(df['time'], df['S_0.01_mM'], '-o', label='S_0.01_mM')
ax.plot(df['time'], df['S_0.003_mM'], '-o', label='S_0.003_mM')
```

```
ax.set_xlabel('Time [s]')
ax.set_ylabel('Concentration [M]')
ax.legend()
plt.show()
```



! Alternative solution: for-loop

An alternative solution is using a for-loop, this is beyond the scope of the course and you will not get exam questions regarding this.

However it is more flexible than writing code for every column - the same code could be used datasets with hundreds or thousands of columns.

```
fig, ax = plt.subplots()

for column_name in df.keys(): # This loops over each column name of the dataframe.
    if column_name != 'time': # Means "if column_name is not equal to 'time'".
        # This only happens when the above condition is met.
        ax.plot(df['time'], df[column_name], '-o', label=column_name)

ax.set_xlabel('Time [s]')
ax.set_ylabel('Concentration [M]')
ax.legend()
plt.show()
```

(c) Finding the slopes

In order to apply the Michaelis Menten equation we need the slopes of these curves as that is the reaction velocity V_0, luckily we have learned how to make fits! So we make a fit to the linear function to determine the slope a in,

$$y(x) = ax + b$$

As always, start by writing the function

```
def linear_function(x, a, b):
    ## Your task: Implement the linear function
    result = ...
    return result
```

```
def linear_function(x, a, b):
    ## Your task: Implement the linear function
    result = a * x + b
    return result
```

Now we want to find the slope for each column, to help with that we will introduce a function that takes the time column and one of the concentration columns and returns the slope

```
def find_slope(time_data, abs_data):

    ## This selects the times and absorbances that have been measured (not NaN)
    ## Don't worry about this - this is just to help you with this exercise.
    indices = np.where(~abs_data.isna())[0]
    time_data = time_data[indices]
    abs_data = abs_data[indices]

    ## Find the slope and the intercept using curve_fit and return the slope
    fitted_parameters, trash = curve_fit(linear_function, time_data, abs_data)
    slope = fitted_parameters[0]
    return slope
```

Now we want to use this for each set of measurements at different substrate concentrations.

```
## Your task: Find the slopes for every substrate concentration.
slope_08 = find_slope(df['time'], df['S_0.8_mM'])
slope_04 = find_slope(df['time'], df['S_0.4_mM'])
slope_02 = find_slope(df['time'], df['S_0.2_mM'])
slope_012 = find_slope(df['time'], df['S_0.12_mM'])
slope_005 = find_slope(df['time'], df['S_0.05_mM'])
slope_002 = find_slope(df['time'], df['S_0.02_mM'])
slope_001 = find_slope(df['time'], df['S_0.01_mM'])
slope_0003 = find_slope(df['time'], df['S_0.003_mM'])

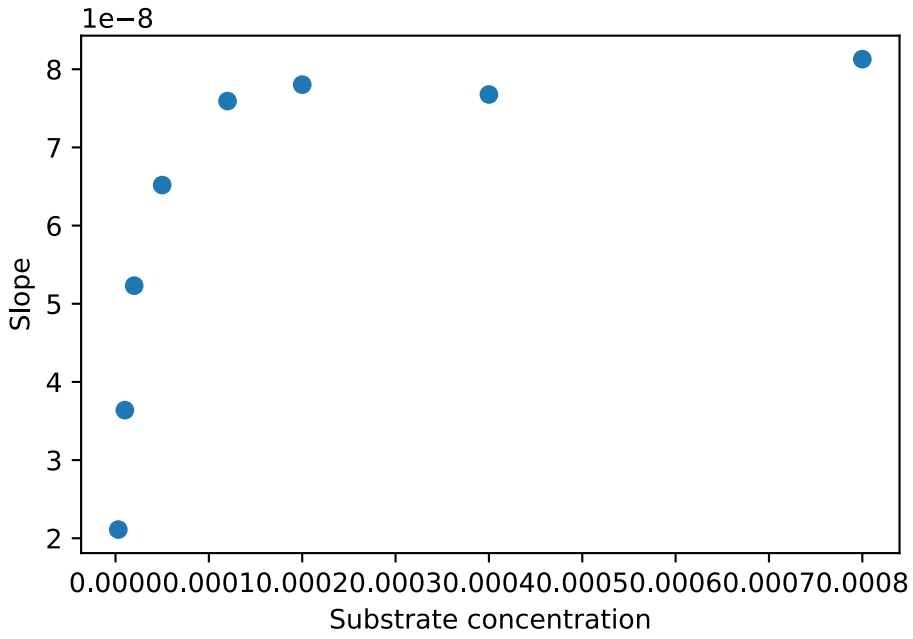
## This gathers the slopes and makes an array with the substrate concentrations.
substrate_concentrations = np.array([0.8, 0.4, 0.2, 0.12, 0.05, 0.02, 0.01, 0.003]) *
10**(-3)
slopes = np.array([slope_08, slope_04, slope_02, slope_012, slope_005, slope_002,
slope_001, slope_0003])
```

Now make a plot of the slopes versus the substrate concentrations using the two arrays `substrate_concentrations` and `slopes`.

```
fig, ax = plt.subplots()

## Your task plot the slopes vs. substrate concentration.
ax.plot(substrate_concentrations, slopes, 'o')

## Sets x and y axis labels.
ax.set_xlabel('Substrate concentration')
ax.set_ylabel('Slope')
plt.show()
```




```
## Your task: Estimate K_M and V_max from the plot.
K_M_estimate = 0.00001
V_max_estimate = 8 * 10**(-8)
```

(d) Michaelis Menten

Now that we have the slopes we can fit to the Michaelis Menten equation, again we need to implement the equation

```
def michaelis_menten(S, K_M, V_max):
    ## Your task: Implement the Michaelis-Menten model.
    result = (V_max * S) / (K_M + S)
    return result
```

Then fit to the slopes we extracted to the Michealis-Menten equation

```
## This is your estimates as the initial guess.
initial_guess = [K_M_estimate, V_max_estimate]

## Your task: Make a fit to the Michaelis-Menten equation & extract K_M and V_max.
fitted_parameters, trash = curve_fit(michaelis_menten, substrate_concentrations, slopes,
initial_guess)
K_M_fit, V_max_fit = fitted_parameters

print(K_M_fit)
print(V_max_fit)
```

```
1.1035718037533613e-05
8.121800807252114e-08
```

- What is the physical meaning of K_M and how does it relate to binding affinity?
- What's the meaning of V_{max}? Why does the curve saturate?

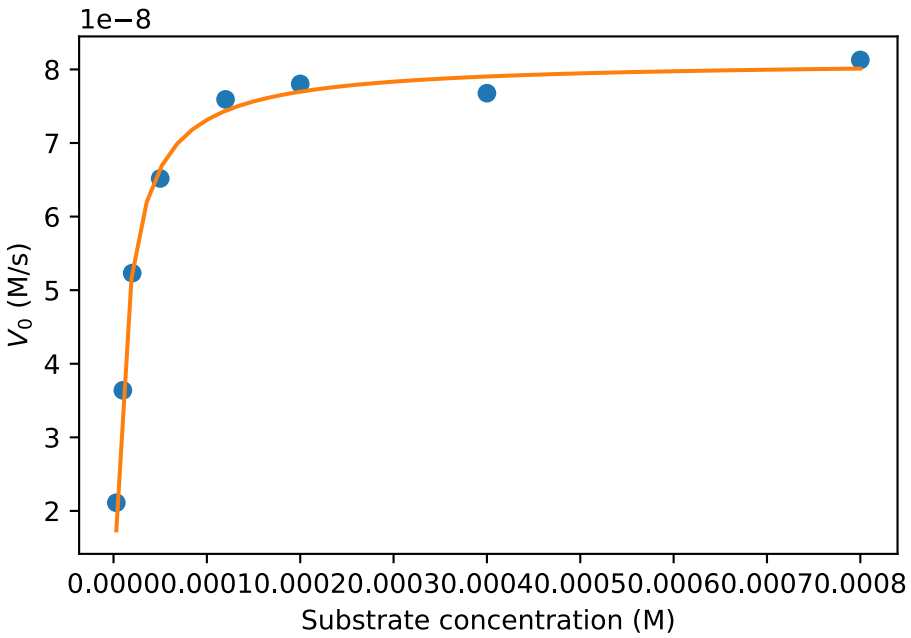
(e) Plot with fit

```
S_smooth = np.linspace(substrate_concentrations.min(), substrate_concentrations.max())
V0_fit = michaelis_menten(S_smooth, K_M_fit, V_max_fit)
```

```
fig, ax = plt.subplots()

## Your task: Plot the data and the fit
ax.plot(substrate_concentrations, slopes, 'o', label='Data')
ax.plot(S_smooth, V0_fit, label='Fit')

## These add x and y axis labels.
ax.set_xlabel('Substrate concentration (M)')
ax.set_ylabel('$V_0$ (M/s)')
plt.show()
```



(f) Turnover number

Given that the enzyme concentration was $3.33 \frac{\text{mg}}{\text{L}}$ and the molecular weight of the enzyme (alkaline phosphatase) is 86000 $\frac{\text{g}}{\text{mol}}$, calculate the turnover number k_{cat}.

Start by converting the enzyme concentration to molar units and then calculate the turnover number.

```
## Your task: Calculate the enzyme concentration and use that and your fit to
## calculate kcat.
enzyme_conc = (3.33*10**(-3)) / 86000
kcat = V_max_fit / enzyme_conc
print(kcat)
```

```
2.097522130401447
```

- What is the biochemical meaning of this number?

---

6 Analysis of a data set obeying Michaelis-Menten kinetics

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
from IPython.display import display
pd.set_option('display.max_rows', 6)
```

The kinetics for an enzyme were investigated using the absorbance of the product to calculate the initial velocities for a range of different initial concentrations of the substrate. The data given in the dataset `analys-data-set-obeyin.xlsx` was obtained. Use these data to answer the following questions.

```
from fysisk_biokemi.datasets import load_dataset
df = load_dataset('week49_2') # Load from package for the solution so it doesn't require to interact.
display(df)
```

	[S]_(mM)	V0_(uM/s)
0	1	166.5
1	1	161.0
2	1	172.0
…	…	…
36	50	478.5
37	50	472.0
38	50	484.0

(a) SI Units

Convert the concentrations of substrate and the velocities to the SI units M and $\mathrm{M} \cdot \mathrm{s}^{-1}$, respectively.

```
## Your task: Add new columns with both properties in proper SI units.
## Use the column names: '[S]_(M)' and 'V0_(M/s)'
df['[S]_(M)'] = df['[S]_(mM)'] * 10**(-3)
df['V0_(M/s)'] = df['V0_(uM/s)'] * 10**(-6)
display(df)
```

	[S]_(mM)	V0_(uM/s)	[S]_(M)	V0_(M/s)
0	1	166.5	0.001	0.000166
1	1	161.0	0.001	0.000161
2	1	172.0	0.001	0.000172
…	…	…	…	…
36	50	478.5	0.050	0.000478

	[S]_(mM)	V0_(uM/s)	[S]_(M)	V0_(M/s)
37	50	472.0	0.050	0.000472
38	50	484.0	0.050	0.000484

(b) Plot & estimate

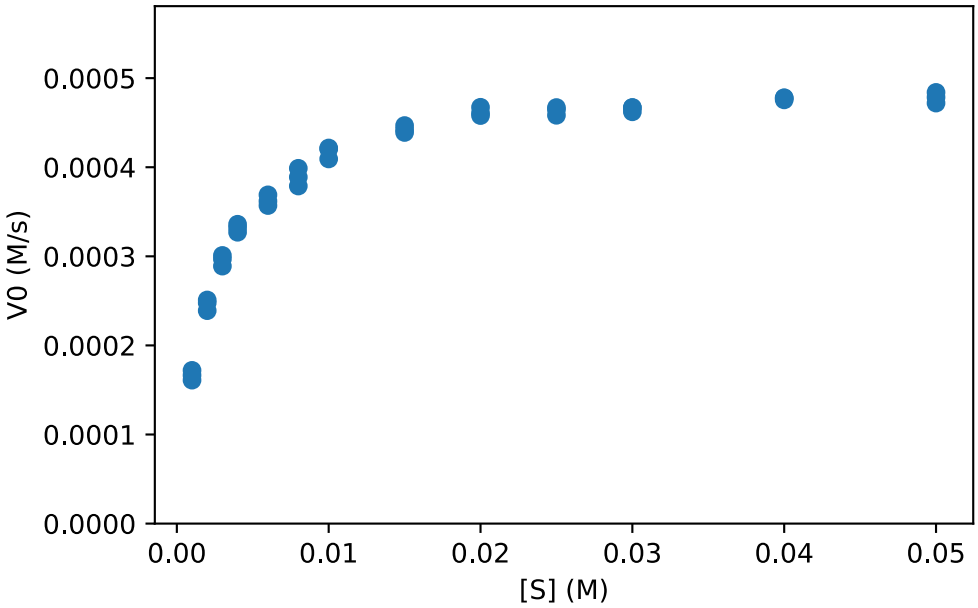
Plot the initial velocities, V_0, as a function of substrate concentrations, $[S]$. Estimate K_M and $V_{\max}$ from this plot.

```
fig, ax = plt.subplots()

## Your task: Plot '[S]_(M)' vs 'V0_(M/s)'.
ax.plot(df['[S]_(M)'], df['V0_(M/s)'], 'o')

## Sets x and y-axis labels:
ax.set_xlabel('[S] (M)')
ax.set_ylabel('V0 (M/s)')

## EXTRA: Sets the y-axis limits.
ax.set_ylim(0, df['V0_(M/s)'].max()*1.2)
```



Put your estimate of K_M and $V_{\max}$ in the cell below:

```
## Task: Put your estimate of the two parameters.
Vmax_estimate = 0.0005
K_M_estimate = 0.005
```

(c) Fit

Now we want to fit using the Michaelis-Menten equation, as per usual when the task is fitting we have to define the function we are fitting with

```
def michaelis_menten(S, K_M, Vmax):
    ## Your task: Implement the Michaelis-Menten equation.
    ## Be careful with parentheses.
    result = (Vmax * S) / (K_M + S)
    return result
```

And then we can follow our usual procedure to make the fit

```
initial_guess = [K_M_estimate, Vmax_estimate]

## Your task: Use curve_fit
fitted_parameters, trash = curve_fit(michaelis_menten, df['[S]_(M)'], df['V0_(M/s)'],
initial_guess)

## Extracts and prints
K_M_fit, Vmax_fit = fitted_parameters
print(Vmax_fit)
print(K_M_fit)
```

```
0.0005007806028375277
0.0020861816467581044
```

How do these values compare to your estimate?

(d) Plot fit & data

Plot the fit alongside the data.

```
## Your task: Evaluate the fit
S_smooth = np.linspace(0, 0.055)
V0_fit = michaelis_menten(S_smooth, K_M_fit, Vmax_estimate)

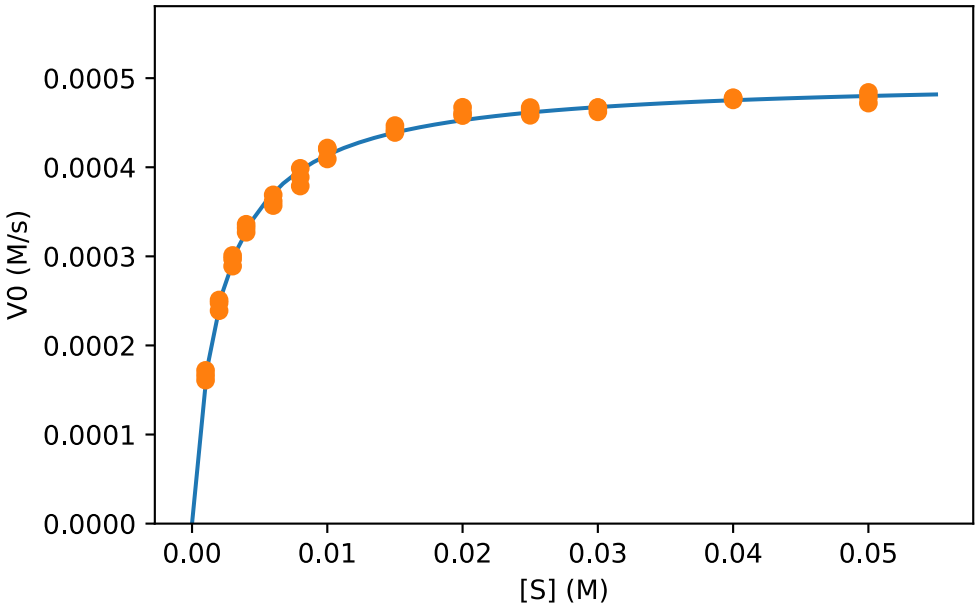
## Your task: Make the plot
fig, ax = plt.subplots()

## Your task: Make plot of the fit.
ax.plot(S_smooth, V0_fit)

## This plot the data and sets the options as before.
ax.plot(df['[S]_(M)'], df['V0_(M/s)'], 'o')

## Sets x and y-axis labels:
ax.set_xlabel('[S] (M)')
ax.set_ylabel('V0 (M/s)')

## EXTRA: Sets the y-axis limits.
ax.set_ylim(0, df['V0_(M/s)'].max()*1.2)
```



(e) k_{cat}

The enzyme concentration was 100 nM use that and your fit to determine k_{cat}.

```
## Your task: Set the enzyme concentration in proper units.
enzyme_concentration = 100 * 10**(-9) # nM

## Your task calculate kcat
kcat = Vmax_fit / enzyme_concentration # (nM/s) / nM  -> 1/s
print(kcat)
```

```
5007.806028375277
```

---

7 The Michaelis Menten equation

Consider a reaction catalyzed by an enzyme obeying the Michaelis-Menten kinetics model.

$$E + S \underset{k_{-1}}{\overset{k_1}{\rightleftharpoons}} ES \overset{k_2}{\rightarrow} E + P$$

$$V_0 = V_{\max} \frac{[S]}{[S] + K_M}$$

(a) Enzyme saturation

Explain what is meant by enzyme saturation.

! Answer

Enzyme saturation simply means the fraction of enzymes in complex with a substrate compared to the total enzyme concentration. Complete saturation occurs when all enzymes is occupied with a substrate molecule.

(b) Enzyme saturation

Calculate the level of enzyme saturation, f_{ES}, at substrate concentrations of

- $[S] = 0.1 \cdot K_M$
- $[S] = 0.5 \cdot K_M$
- $[S] = 1 \cdot K_M$
- $[S] = 10 \cdot K_M$
- $[S] = 100 \cdot K_M$

```
## Your task calculate the degree of enzyme saturation for the prescribed cases
f_ES_1 = 0.1 / (1 + 0.1)
f_ES_2 = 0.5 / (1 + 0.5)
f_ES_3 = 1 / (1 + 1)
f_ES_4 = 10 / (1 + 10)
f_ES_5 = 100 / (1 + 100)

print(f_ES_1, f_ES_2, f_ES_3, f_ES_4, f_ES_5)
```

```
0.09090909090909091 0.3333333333333333 0.5 0.9090909090909091 0.9900990099009901
```

```
## An alternative solution using a function
def enzyme_saturation(x):
    return x / (1 + x)

## Then use the function for each calculation
f_ES_1 = enzyme_saturation(0.1)
f_ES_2 = enzyme_saturation(0.5)
f_ES_3 = enzyme_saturation(1)
f_ES_4 = enzyme_saturation(10)
```

```
f_ES_5 = enzyme_saturation(100)
print(f_ES_1, f_ES_2, f_ES_3, f_ES_4, f_ES_5)
```

```
0.09090909090909091 0.3333333333333333 0.5 0.9090909090909091 0.9900990099009901
```

! Answer

For the Michealis-Menten kinetics model the following holds true

$$[ES] = \frac{([E]_T - [ES])[S]}{K_M}$$

$$[ES] = [E]_T \frac{[S]}{[S] + K_M}$$

$$f_{ES} = \frac{[ES]}{[E]_T} = \frac{[S]}{[S] + K_M}$$

In cases like here where $[S]$ is given in terms of K_M, e.g. $[S] = x \cdot K_M$, we can calculate $f(ES)$ as

$$f_{ES} = \frac{[ES]}{[E]_T} = \frac{xK_M}{xK_M + K_M} = \frac{xK_M}{K_M(1+x)} = \frac{x}{1+x}$$

(c) Compare K_M and K_D.

The Michaelis-Menten constant K_M is defined as:

$$K_M = \frac{k_{-1} + k_2}{k_1}$$

In a specific reaction, the following rate constants were determined:

- $k_1 = 7 \cdot 10^7 \cdot \mathrm{M}^{-1} \cdot \mathrm{s}^{-1}$,
- $k_{-1} = 8 \cdot 10^5\ \mathrm{s}^{-1}$
- $k_2 = 10^3\ \mathrm{s}^{-1}$

Calculate the values of K_M and K_D (The dissociation constant of the enzyme:substrate complex).

Does K_M approximate the dissociation constant for the ES complex in this case, and how can this be estimated directly from the values of the microscopic rate constants?

```
k1 = 7 * 10**7
k2 = 10**3
k_m_1 = 8 * 10**5

K_M = (k_m_1 + k2) / k1
K_D = k_m_1 / k1
```

```
print(K_M)
print(K_D)
```

```
0.011442857142857144
0.011428571428571429
```

! Answer

Yes, since $k_{-1} \gg k_2$ it follows that K_M and K_D for the ES complex are almost identical.

(d) Calculate k_2

Assume that a solution of an enzyme at a concentration of $1 \cdot 10^{-7}$ M with a substrate concentration of $[S] = 100 \cdot K_M$ has $V_0 = 10^{-4}\,\mathrm{M} \cdot \mathrm{s}^{-1}$

Given this information and with your answers to (b) and (c) in mind, approximate the constant k_2. What is this constant also called?

```
## Your task: Assign known values
V0 = 1 * 10**(-4) # M / s
E_tot = 1 * 10**(-7) # M

## Your task: Once you have figured out how to calculate k2, use Python to calculate it.
Vmax = V0 / f_ES_5
k2 = Vmax / E_tot
print(k2)
```

```
1010.0000000000001
```

! Answer

This constant is also known as k_{cat}.

(e) Lineweaver-Burk plot

The values of $V_{\max}$ and K_M have historically been determined from a Lineweaver-Burk plot.

How does the x- and y-intercepts in a Lineweaver-Burk plot relate to $V_{\max}$ and K_M?

! Answer

